can u link claude with a langgraph agent

You can absolutely link Claude (Anthropic) with a LangGraph agent. LangGraph is model-agnostic, meaning you can plug in any LLM integration supported by LangChain. For Claude, you will use the `langchain-anthropic` package.

Here is a step-by-step implementation showing how to hook Claude up to a stateful LangGraph agent that can call external tools.

The Implementation

1. Prerequisites

First, make sure you have the required packages installed and your API key exported:

Bash

```bash
pip install -U langgraph langchain-anthropic
export ANTHROPIC_API_KEY="your-api-key-here"
```

2. Building the Agent Loop

The typical structure involves defining your graph state, initializing Claude with tool capabilities, creating an execution node, and setting up conditional routing.

① **Define the State & Model**

Initialize Claude

Define the shared state (the conversation history) and initialize the `ChatAnthropic` model.

Python

```python
import os
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_anthropic import ChatAnthropic
```

```
from langchain_core.tools import tool

# Define the shared state structure
class State(TypedDict):
    messages: Annotated[list, add_messages]

# Initialize Claude
llm = ChatAnthropic(model="claude-3-5-sonnet-20240620")
```

② Create and Bind Tools

Augment Claude's capabilities

Create a tool (a simple Python function with docstrings) and bind it to Claude so the model knows how to format tool arguments.

Python

```
@tool
def get_weather(location: str) -> str:
    """Get the current weather for a given location."""
    if "london" in location.lower():
        return "It's 15°C and raining in London."
    return "It's 22°C and sunny."

tools = [get_weather]
# Bind tools to Claude
llm_with_tools = llm.bind_tools(tools)
```

③ Define Graph Nodes

Logic functions

Create a chatbot node that runs the model and outputs the state updates.

Python

```
def chatbot(state: State):
    return {"messages": [llm_with_tools.invoke(state["messages"])]}
```

④ Assemble and Compile the Graph

Build the state machine

Wire up the components into a graph. You can use LangGraph's pre-built `ToolNode` to easily handle running your functions if Claude decides to call them.

Python

```python
from langgraph.prebuilt import ToolNode, tools_condition

# 1. Initialize graph builder
builder = StateGraph(State)

# 2. Add nodes
builder.add_node("chatbot", chatbot)
builder.add_node("tools", ToolNode(tools))

# 3. Define edges and routing
builder.add_edge(START, "chatbot")

# If chatbot calls a tool, route to "tools". Otherwise, route to END.
builder.add_conditional_edges("chatbot", tools_condition)
builder.add_edge("tools", "chatbot")

# 4. Compile
graph = builder.compile()
```

3. Testing the Agent

Once compiled, you can stream messages through the graph. If you ask about the weather, Claude will generate a `tool_call`, the `ToolNode` will intercept and run it, and Claude will read the final output to formulate a response:

GitHub

Python

```python
inputs = {"messages": [("user", "What's the weather like in London?")]}
for event in graph.stream(inputs):
    for value in event.values():
        print(value["messages"][-1].content)
```

Why Claude thrives in LangGraph: Claude excels at complex workflows due to its precision in tool calling and structure parsing. When building multi-agent architectures, its ability to strictly adhere to system formatting boundaries keeps state machines from breaking down.

For a deeper dive into the orchestration mechanics between LangGraph and Claude patterns, you can check out this guide on Building Effective Agents with LangGraph which breaks down prompt chaining, routing, and tool-handling behaviors.

what is claude

Claude is a family of large language models (LLMs) developed by **Anthropic**, an AI safety and research company founded in 2021 by former leaders from OpenAI.



Claude by Anthropic. Source: Credo AI

Key Characteristics of Claude

Claude is designed to be a highly capable conversational assistant, programmer, and analytical partner. It stands out in the AI landscape for a few specific reasons:

- **Constitutional AI:** Unlike models trained purely on human feedback, Anthropic trains Claude using a method called "Constitutional AI." The model is given a set of written principles (a "constitution" based on documents like the UN Declaration of Human Rights) and effectively critiques its own behavior to ensure it stays helpful, harmless, and honest.
- **Advanced Coding & Logic:** Claude models (particularly the *Sonnet* and *Opus* tiers) are widely recognized for exceptional performance in software engineering, complex logic, multi-step math problem solving, and generating clean code.

- **Large Context Windows:** Claude was an early pioneer in handling massive amounts of data at once, allowing users to upload entire books, complex codebases, or hundreds of pages of financial documents to analyze in a single prompt.
- **Artifacts Feature:** In its user interface, Claude can open a dedicated side window ("Artifacts") to display, live-preview, and let users interact with generated code, SVG graphics, websites, or markdown documents separate from the main chat.

The Model Lineup

Anthropic organizes the Claude family into three primary tiers, designed to balance speed, cost, and capability:

Model Tier	Purpose & Description
Claude Haiku	The fastest and most cost-effective model. Built for lightweight tasks, quick actions, and high-volume data processing.
Claude Sonnet	The mainstream powerhouse. It strikes an optimal balance between high intelligence and speed, making it the default choice for coding and complex analysis.
Claude Opus	The top-of-the-line model designed for deep reasoning, highly abstract strategy, and tackling the most complex, open-ended problems.

...

does it have offline models

No, **Anthropic does not offer offline versions of Claude** that you can download and run directly on your own hardware. The actual Claude models (like Claude 3.5 Sonnet or Opus) are proprietary, closed-source, and too massive to run locally on a standard consumer computer; they require Anthropic's cloud infrastructure (or cloud partners like AWS Bedrock and Google Cloud).

However, **there is a major exception regarding Claude's tooling:**

Anthropic allows you to detach their developer tools—specifically **Claude Code** (their command-line agentic coding assistant) and **Claude Desktop**—from their cloud servers and point them at **local, offline open-weight models** running on your own machine.

Eigent AI

If you want to use Claude's ecosystem completely offline and for free, you can configure it to run using local models through the following approaches:

1. Claude Code with Local Models (Ollama / Unsloth / LM Studio)

The `claude` CLI tool allows you to swap out the backend API URL. By changing the base URL, you can route Claude's advanced developer agent harness to local models running via Ollama, LM Studio, or llama.cpp.

Medium

To do this, you load a highly capable local coding model (like `Qwen-2.5-Coder` or `Devstral`) on your machine and redirect the CLI:

Bash

```
# 1. Start your local model server (e.g., via Ollama on port 11434)
ollama run qwen2.5-coder:14b

# 2. Redirect Claude Code to your localhost
export ANTHROPIC_BASE_URL="http://localhost:11434/v1"

# 3. Launch Claude Code using your local model
claude --model qwen2.5-coder:14b
```

2. Claude Desktop Third-Party Inference

Claude Desktop features a **Third-Party Inference** setting. This allows users to completely log out of their Anthropic cloud account and route the desktop application's workspace, skills, and Model Context Protocol (MCP) tools through a local gateway (like an OpenAI-compatible proxy running LiteLLM or Ollama).

Eigent AI

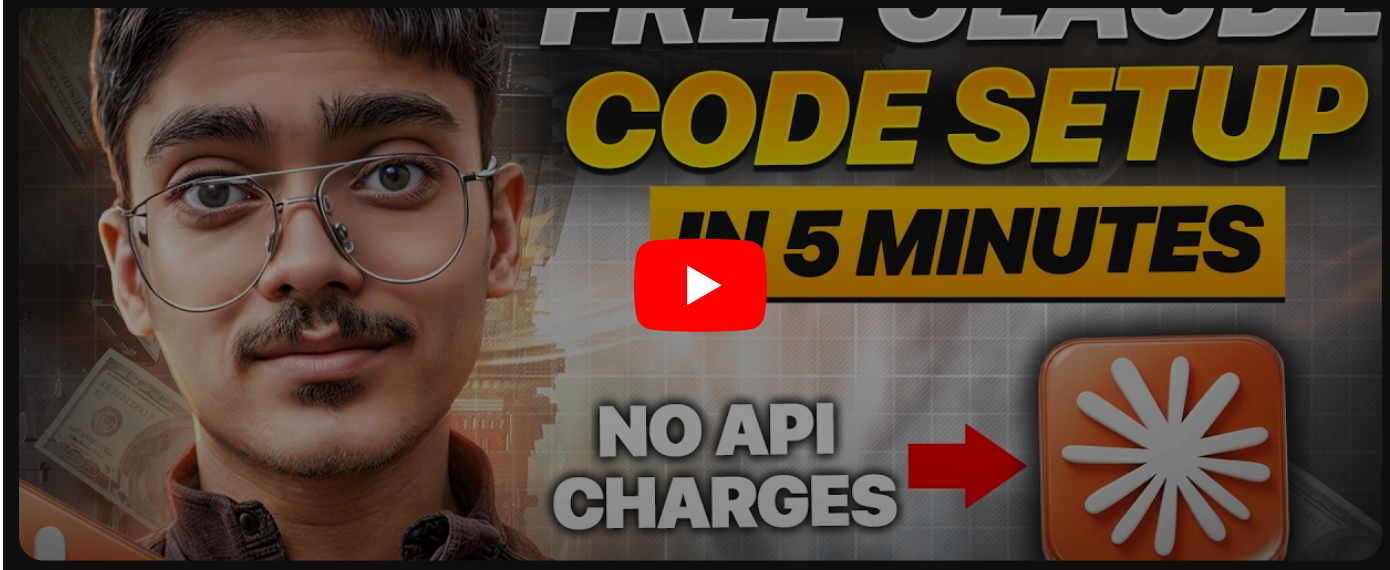
Summary

- **The Claude Models: ❌ No.** They are strictly cloud-based (via Anthropic API, AWS, or Azure).
- **The Claude App/Agent Harness: Yes.** You can run the application frameworks completely offline by pairing them with local, open-source models.

  Reddit

To get a visual guide on setting this up, a tutorial like Claude Code Running FREE & Offline walkthrough provides step-by-step terminal commands for configuring Ollama and Node.js to use the Claude environment completely without an internet connection.

Claude Code Running FREE & Offline – Full Tutorial Inside - YouTube
Shreyas Raj · 22K views



can i use the claude desktop chat UI with my langgraph agent as its UI

Yes, you absolutely can! However, there is a fundamental inversion in how the relationship works.

Instead of wrapping the Claude UI *around* your LangGraph agent, you expose your LangGraph agent as a tool *to* Claude Desktop using Anthropic's **Model Context Protocol (MCP)**.

Memgraph

In this setup, Claude Desktop acts as the frontend interface and the primary "Brain." When you ask a question that requires your custom workflow, Claude forwards the request to your LangGraph agent (the "Secondary Brain/Executor") via an MCP pipe, runs the graph, and displays the final response right inside the native desktop chat window.

The Architecture

- **The UI:** Claude Desktop (Speaks JSON-RPC via MCP).

Memgraph

- **The Bridge:** A `FastMCP` Python server that registers your agent as a tool.
- **The Brain:** Your custom LangGraph compiled state machine.
  Medium

Step-by-Step Implementation

1. Install Dependencies

You need the standard `mcp` SDK alongside `langchain-mcp-adapters` to bridge the gap between LangGraph and the protocol.

Bash

```
pip install fastmcp langgraph langchain-anthropic
```

2. Wrap your LangGraph Agent in an MCP Server Generect

Create a file named `my_agent_mcp.py`. This script sets up a FastMCP server and wraps your graph invocation inside a Python function decorated with `@mcp.tool()`.

Python

```python
# my_agent_mcp.py
import os
from fastmcp import FastMCP
from langgraph.graph import StateGraph, START
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages
from langchain_anthropic import ChatAnthropic

# ---- 1. Initialize your LangGraph Agent ----
class State(TypedDict):
    messages: Annotated[list, add_messages]

# Simple example graph node
def agent_node(state: State):
    llm = ChatAnthropic(model="claude-3-5-sonnet-20240620")
    # Your complex agent logic, RAG retrieval, or tool calling goes here
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

builder = StateGraph(State)
builder.add_node("agent", agent_node)
```

```python
builder.add_edge(START, "agent")
graph = builder.compile()

# ---- 2. Initialize FastMCP Server ----
mcp = FastMCP("LangGraph_Agent")

@mcp.tool()
async def query_my_agent(user_prompt: str) -> str:
    """
    Use this tool to trigger the backend stateful LangGraph agent.
    Describe what you want the custom agent workflow to process.
    """
    # Invoke your LangGraph state machine
    inputs = {"messages": [("user", user_prompt)]}
    result = await graph.ainvoke(inputs)

    # Extract the final message content from the state
    final_message = result["messages"][-1].content
    return f"LangGraph Agent Output:\n{final_message}"

if __name__ == "__main__":
    mcp.run()
```

3. Register the Agent with Claude Desktop

You need to tell the Claude Desktop application where to find your script. Open your `claude_desktop_config.json` file.

- **Mac:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

Add your agent server under the `mcpServers` block:

JSON

```json
{
  "mcpServers": {
    "my-langgraph-agent": {
      "command": "python",
      "args": [
        "/absolute/path/to/your/my_agent_mcp.py"
      ],
      "env": {
        "ANTHROPIC_API_KEY": "your-api-key-here"
      }
    }
```

```
    }
  }
```

(Make sure to use the absolute path to your script, and verify that `python` is available in your global system environment path).

4. Talk to your Agent in Claude Desktop

1. Completely quit and restart the **Claude Desktop** app.
2. Look at the bottom right corner of the chat box. You should see a small **plug icon** 🔌. If you hover over it, you'll see `LangGraph_Agent` listed with the `query_my_agent` tool active.
3. Simply type a message like: *"Run my custom agent and ask it to analyze our database for Q3 trends."*

Claude will recognize that it needs the tool, display a small loading block showing that it's hitting `query_my_agent`, run your full underlying Python graph topology, and print out the state updates cleanly inside your chat window!

can ollama desktop running qwen models run MCP servers

Yes, but with an important distinction: **the raw Ollama desktop engine cannot natively host an MCP server configuration out of the box.**

To make it work, you use a **host or client script** to connect your local Qwen model to your MCP servers. The **Qwen lineup (especially Qwen 2.5 and newer tiers)** is highly optimized for this because it has exceptional native tool-calling capabilities, which are required to talk to MCP infrastructure.

Medium +1

Depending on how you want to use it, there are two primary methods for running MCP servers with local Qwen models via Ollama.

Method 1: Using an Open-Source MCP Host (The Easiest Route)

Because Ollama doesn't have a built-in `config.json` system for MCP like Claude Desktop does, developers use tiny local orchestration tools like `mcphost` or terminal clients like `mcp-client-for-ollama` (`ollmcp`) to act as the middleman.

Using a tool like `mcphost` (Go-based) or `ollmcp` (Python-based), you can point a local configuration file directly to your Ollama Qwen instance:

① Pull your Qwen Model

Ollama Terminal

Make sure you have a tool-calling capable Qwen model downloaded locally.

Bash

```
ollama pull qwen3.5:4b
```

② Create your MCP Config

local.json

Create a configuration file listing the MCP servers you want Qwen to access (e.g., a local SQLite database or a Filesystem server).

JSON

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path/to/secure/
    }
  }
}
```

③ Launch the Host

Connect everything

Run an MCP hosting manager to bridge Ollama's Qwen model with your local server configurations.

Bash

```
# Example using mcphost
mcphost -m ollama:qwen3.5:4b --config "local.json"
```

Method 2: Within Code (LangChain / LangGraph Adapter)

If you are writing custom Python applications (like the LangGraph agent built previously), you can use the `langchain-mcp-adapters` package to bind your local Ollama Qwen instance to any standard stdio or SSE MCP server.

Python

```python
from langchain_ollama import ChatOllama
from langchain_mcp_adapters.tools import load_mcp_tools

# 1. Initialize local Qwen model via Ollama
model = ChatOllama(model="qwen3.5:4b", temperature=0)

# 2. Automatically load tools directly from a running MCP server
# (Like a filesystem server running over Node stdio)
mcp_tools = load_mcp_tools(
    "stdio",
    command="npx",
    args=["-y", "@modelcontextprotocol/server-filesystem", "/your/dir"]
)

# 3. Bind the MCP tools natively into Qwen's engine
model_with_mcp = model.bind_tools(mcp_tools)
```

> **Why Qwen works so well here:** Many smaller local open-weight models struggle to format JSON correctly when an MCP server demands multi-parameter tool execution. The Qwen models have been specifically fine-tuned on dense function-calling datasets, making them the most stable local option for parsing complex MCP server interactions.

how do i install mcphost

`mcphost` is an open-source, terminal-based host application written in Go. It connects local models (like Ollama/Qwen) or cloud APIs (like Claude/Gemini) to your custom Model Context Protocol (MCP) servers.

Go Packages

Depending on your operating system, you have two primary ways to install it.

Option 1: Using Homebrew (macOS / Linux)

If you use Homebrew, `mcphost` is available as a standard formula. Run the following command in your terminal:

Bash

```
brew install mcphost
```

Option 2: Using Go (Cross-Platform)

Because it is a Go binary, you can install it on any machine (Windows, macOS, or Linux) that has Go (version 1.23 or newer) installed:

ClickHouse

Bash

```
go install github.com/mark3labs/mcphost@latest
```

(Note: If the `mcphost` command isn't recognized immediately after running this, make sure your global Go binary path—usually `~/go/bin`—is added to your system's `PATH` environment variables).

ClickHouse

Verifying the Installation & Setup

Once installed, you can verify it works by checking the help menu:

Bash

```
mcphost --help
```

Initial Configuration

When you run `mcphost` for the first time, it automatically looks for or creates a configuration file located at `~/.mcp.json`. You populate this file with the MCP tools you want your local Qwen models to use:

JSON

```
{
  "mcpServers": {
```

```
    "sqlite": {
      "command": "uvx",
      "args": [
        "mcp-server-sqlite",
        "--db-path",
        "/tmp/my_database.db"
      ]
    },
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/tmp"
      ]
    }
  }
}
```

Launching with your Local Model

With your configuration saved and an Ollama backend active, you can launch `mcphost` directly tied to a local Qwen instance:

Bash

```
mcphost --model ollama:qwen2.5-coder:14b
```

Once inside the interactive terminal session, you can use internal slash commands like `/tools` to see your running MCP configurations or `/quit` to close out the session.